RC CAR TỰ HÀNH 1/16 SCALE

Kế hoạch phát triển Phase 1 → Phase 4

apollomotion (Phi) & Nhã

ĐH Giao thông Vận tải TP.HCM | 2026

Tài liệu nội bộ — Luận văn tốt nghiệp

---

1. Tổng quan dự án

Mục tiêu xây dựng xe RC tự hành 1/16 scale làm luận văn tốt nghiệp, hướng tới sản phẩm thương mại cho doanh nghiệp, đại học, sinh viên mới.

Nguyên tắc thiết kế xuyên suốt

- **Build once, reuse forever — hardware modular (JST connector), software parameter-driven**
- Make it work first, optimize later
- Thay track/môi trường → chỉ chỉnh perception.yaml (BEV corners)
- Thay xe → chỉnh robot.yaml
- Sim và hardware phát triển song song

Phân công

Phi (apollomotion)	Nhã
Perception pipeline	NMPC optimization
EKF state estimation	Velocity control
SLAM + Localization	Nucleo firmware
Nav2 integration	Constraint tuning
GUI Dashboard	Hardware motor control
Luận văn (chính)	Đóng góp chương 3, 4

---

2. Roadmap tổng thể (12 tuần)

Tuần	Phase	Mục tiêu	Trạng thái
1-2	Phase 1: Simulation	Perception + NMPC + EKF trong Gazebo	✅ XONG
3-4	Phase 1.5: Sim tracks + HW prep	3 track sim + camera Pi deploy	🔄 TIẾP THEO
5-6	Phase 1.5: Hardware lane following	Xe bám lane track thật 0.3 m/s	📅 KẾ HOẠCH
7-8	Phase 2: SLAM	Visual odometry + lane-based map	📅 KẾ HOẠCH
9-10	Phase 3: Navigation	Nav2 + obstacle avoidance	📅 KẾ HOẠCH
11	Phase 4: GUI	Foxglove dashboard	📅 KẾ HOẠCH
12	Luận văn + Demo	Video + demo live	📅 KẾ HOẠCH

---

3. Phase 1 — Simulation ✅ XONG

3.1 Pipeline đã hoàn chỉnh

Pipeline hiện tại là fully closed loop:

- Camera (30Hz) → Perception → e_y, e_psi, kappa → NMPC
- IMU (100Hz) + Joint States → EKF → velocity feedback → NMPC
- NMPC (acados RTI) → delta, v_ref → Vehicle Controller → Gazebo

3.2 Kết quả đạt được

- [x] Perception: BEV → Sliding Window → Cubic poly + arc-length @ 30Hz
- [x] EKF: Camera + IMU + Wheel encoder, ekf_healthy=true @ 100Hz
- [x] NMPC Kloeser 2020: acados RTI, solve time 1-5ms @ 30Hz
- [x] Curvature-based speed reduction: v_ref = V_MAX / (1 + 8*|kappa|)
- [x] EKF velocity feedback thay first-order model
- [x] Reset node: Q+Enter manual reset, auto reset khi lane lost
- [x] IMU plugin Gazebo Harmonic + bridge ROS2
- [x] VehicleState.msg và LaneState.msg custom messages

3.3 Tracks simulation

- [x] Track 1: Oval đơn giản (lane_track.sdf, 9x2m)
- [ ] Track 2: Track phức tạp (track_test.sdf, 6x6m, có chicane + S-curve)
- [ ] Track 3: Track với vật cản

3.4 Tham số NMPC hiện tại

Tham số	Giá trị	Ghi chú
V_MAX	2.0 m/s	Sim; hardware giai đoạn 1: 0.3 m/s
N (horizon)	30 steps	Tf=1.0s, dt=33ms
W_N (lateral)	200.0	Penalize lane offset
W_ALPHA (heading)	100.0	Penalize heading error
W_DELTA (steering)	10.0	Smooth steering
kappa_factor	8.0	Speed reduction tại cua
Solve time	1-5ms	Margin 28ms so với 33ms period

---

4. Phase 1.5 — Deploy Hardware 🔄 TIẾP THEO

4.1 Mục tiêu

Xe bám lane track thật ở 0.3 m/s. Chỉ điều khiển góc lái — tốc độ cố định thấp.

Pipeline deploy đơn giản (không cần EKF giai đoạn đầu):

- Pi Camera → perception_node (Pi) → /lane_state → [ROS2 network]
- mpc_node (laptop) → /steering_angle, /velocity → [ROS2 network] → Nucleo → Servo + ESC

4.2 Sim tracks cần hoàn thành

- [ ] Track 2 (track_test.sdf): spawn đúng vị trí, test full pipeline
- [ ] Track 3: thiết kế với vật cản, test perception + NMPC
- [ ] Tune NMPC weights cho từng track
- [ ] Record video demo 3 tracks trong sim

4.3 Hardware checklist

- [ ] Giải quyết Pi Camera 3 driver (picamera2 hoặc libcamera)
- [ ] Calibrate camera intrinsics bằng checkerboard
- [ ] Recalibrate BEV homography (SRC/DST) với Pi Camera 3 Wide thật
- [ ] Test perception node trên Pi, stream /camera/image_raw về laptop
- [ ] Tune AdaptiveThreshold cho băng keo đen trên nền trắng
- [ ] Verify NMPC nhận /perception/lane_state từ Pi qua network
- [ ] Test ở 0.3 m/s trên đường thẳng trước
- [ ] Test cua nhẹ sau khi đường thẳng ổn định
- [ ] Record video milestone: xe bám lane track thật

4.4 Track thật — Thông số

Thông số	Giá trị
Nền	Giấy trắng A0 (4-8 tờ ghép)
Lane marking	Băng keo đen, rộng 2-3cm
Lane width	50cm (2 dải song song)
Kích thước tối đa	~3.4m x 2.4m (8 tờ A0)
Số track thật	3 track (đơn giản → phức tạp → có vật cản)

4.5 Hardware Architecture

Nguyên tắc: JST connector cho mọi kết nối, fuse riêng từng module, Nucleo update qua USB không cần tháo xe.

Component	Model	Interface
Compute	Raspberry Pi 5	ROS2 Jazzy (Docker), IP: 192.168.1.23

Component	Model	Interface
Camera	Pi Camera 3 Wide	CSI, IMX708, 30fps
IMU	BNO055	I2C → Nucleo → USB Serial (/dev/ttyACM0)
MCU	Nucleo F411RE	Servo PWM + ESC PWM + BNO055 I2C
Steering	Servo	PWM từ Nucleo
Drive	ESC + Motor	PWM từ Nucleo
Power	LiPo 3S	5V reg → Pi, 6V reg → Servo

---

5. Phase 2 — SLAM & Localization 📅 Tuần 7-8

5.1 Mục tiêu

Xe build map realtime từ camera, biết vị trí của mình trên map. Không dùng LiDAR — môi trường vạch kẻ đường cho phép dùng visual approach.

5.2 Technical approach

- Visual Odometry: estimate motion giữa các frame liên tiếp
- Lane Graph Map: map = graph của lane centerline (nodes + edges), lưu JSON
- Particle Filter Localization: localize xe trên lane graph

Lý do chọn approach này: môi trường băng keo đen trên nền trắng = contrast cao, lane geometry rõ ràng → không cần feature-based SLAM phức tạp.

5.3 Sim checklist

- [ ] Thêm ground truth odometry từ Gazebo để validate VO
- [ ] Implement visual odometry node
- [ ] Build lane graph từ perception output trong sim
- [ ] Implement particle filter localization
- [ ] Visualize map + xe position trong RViz/Foxglove

5.4 Hardware checklist

- [ ] Test visual odometry trên Pi Camera 3 thật
- [ ] Build map track thật 1 và 2
- [ ] Validate localization accuracy
- [ ] Record video: xe build map realtime

---

6. Phase 3 — Navigation & Obstacle Avoidance 📅 Tuần 9-10

6.1 Mục tiêu

Xe tự navigate từ điểm A đến điểm B, tránh vật cản trên đường. Full autonomy không cần can thiệp.

6.2 Stack kỹ thuật

- ROS2 Nav2: map_server + amcl + bt_navigator
- NMPC làm local controller (thay thế DWB mặc định của Nav2)
- Obstacle detection: camera-based (băng keo hoặc object đặt trên track)

6.3 Interface Nav2 → NMPC

- Nav2 output: /cmd_vel (linear.x, angular.z)

- Convert cmd_vel → NMPC reference trajectory
- NMPC output: /steering_angle, /velocity

6.4 Sim checklist

- [ ] Thêm obstacle model vào track_test.sdf
- [ ] Test Nav2 với lane map từ Phase 2
- [ ] Integrate NMPC làm Nav2 local controller
- [ ] Test obstacle avoidance trong sim

6.5 Hardware checklist

- [ ] Test full autonomy track thật 2 (không có vật cản)
- [ ] Test obstacle avoidance track thật 3
- [ ] Tune behavior khi gặp vật cản (dừng lại hay vòng qua)
- [ ] Record video: xe tự navigate + tránh vật cản

---

7. Phase 4 — GUI Dashboard 📅 Tuần 11

7.1 Mục tiêu

Dashboard trực quan cho non-technical audience: doanh nghiệp, đại học, sinh viên mới. Người dùng không cần hiểu ROS2 để vận hành xe.

7.2 Stack đề xuất

- Foxglove Studio + custom panels (primary)
- rosbridge_suite + React web app (alternative, chạy trên browser bất kỳ)

7.3 Features cần có

- Camera feed realtime
- Map visualization + vị trí xe
- Vehicle status card: tốc độ, góc lái, mode (manual/auto)
- Mission control: set goal, start/stop, reset xe
- Alert system: lane lost, obstacle detected, EKF unhealthy
- Parameter tuning panel: V_MAX, weights (nice-to-have)

7.4 Checklist

- [ ] Setup Foxglove Studio + rosbridge
- [ ] Camera feed panel
- [ ] Map + vehicle position panel
- [ ] Vehicle status dashboard
- [ ] Mission control (start/stop/reset)
- [ ] Alert system
- [ ] Test toàn bộ GUI với hardware thật

---

8. Luận văn & Demo 📅 Tuần 12

8.1 Cấu trúc luận văn

- Chương 1: Tổng quan, motivation, scope
- Chương 2: Lý thuyết (NMPC Kloeser 2020, EKF, Visual SLAM, Nav2)
- Chương 3: System design + Hardware architecture
- Chương 4: Implementation (Perception, EKF, NMPC, SLAM, Nav2, GUI)
- Chương 5: Kết quả thực nghiệm (sim + hardware, metrics)
- Chương 6: Kết luận + hướng phát triển

8.2 Video demo checklist

- [ ] Video sim: 3 tracks (oval → phức tạp → có vật cản)
- [ ] Video hardware: lane following đường thẳng

- [ ] Video hardware: lane following track phức tạp
- [ ] Video hardware: full autonomy (SLAM + Nav2)
- [ ] Video GUI: dashboard demo

8.3 Demo live checklist

- [ ] Hardware ổn định, không crash
- [ ] Chuẩn bị track thật sẵn sàng
- [ ] Slide deck cho audience doanh nghiệp/đại học
- [ ] Backup plan: video nếu hardware fail
- [ ] GUI demo trên laptop/tablet

---

9. Rủi ro & Phòng ngừa

Rủi ro	Xác suất	Phòng ngừa
Pi Camera 3 driver không ổn định	Cao	Fallback: USB camera hoặc picamera2
Visual SLAM drift trên track thật	Trung bình	Dùng pre-built map + AMCL thay realtime SLAM
Nav2 + NMPC integration phức tạp	Trung bình	Fallback: Nav2 dùng DWB controller thay NMPC
Luận văn không kịp deadline	Cao	Viết song song từ tuần 3, không để dồn cuối
Hardware mechanical failure	Thấp	Mua spare parts trước, JST connector dễ swap
Timeline quá tight	Cao	Scope reduction: Phase 4 GUI có thể đơn giản hóa

---

10. Software Architecture — Parameter-driven

10.1 Config files

Tất cả thông số quan trọng được externalize ra yaml — không hardcode trong code:

- perception.yaml: BEV corners, threshold, sliding window params
- nmpc.yaml: weights, constraints, horizon, vehicle params
- ekf.yaml: Q, R matrices, noise params (TODO)
- slam.yaml: VO params, map resolution (TODO)
- nav2_params.yaml: Nav2 full config (TODO)

10.2 Quy tắc deploy

- Sim → Hardware: chỉ cần recalibrate perception.yaml (BEV corners)
- Track đơn giản → Track phức tạp: tune nmpc.yaml (weights, V_MAX)
- Xe scale khác: sửa robot.yaml (wheelbase, wheel_radius)

- Môi trường ánh sáng khác: tune thresh_block_size, thresh_c trong perception.yaml

10.3 ROS2 network (Laptop ↔ Pi)

Topic	Direction	Node
/camera/image_raw	Pi → Laptop	perception_node (Pi)
/perception/lane_state	Pi → Laptop	mpc_node (Laptop)
/imu/data	Pi → Laptop	ekf_node (Laptop)
/ekf/vehicle_state	Laptop → Laptop	mpc_node
/steering_angle	Laptop → Pi	vehicle_controller (Pi)
/velocity	Laptop → Pi	vehicle_controller (Pi)